02157 Functional Programming
Week 2:
Functions, Types, and Lists

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

September 11, 2026

Outline

1. **Recap: What we have seen of F# so far**
2. Last Week's Exercises
3. Intro to Types
4. Intro to Higher-Order Functions
5. Lists
6. On functional decomposition
7. Exercises

Syntax, Type Checking, and Semantics

- The syntax of a programming language describes what sequences of characters consitute a correctly formed program,
  - ★ e.g. balancing of opening and closing parentheses.
- Type checking checks that all values (including functions) used in the program are indeed defined and used with the correct types,
  - ★ e.g. a function that expects an int cannot be called with a string.
- Semantics says what a syntactically correct, well-typed program means.
  - ★ e.g. `if x>0 then x else 2*x` with $[x \mapsto -3]$ evaluates to -6.
- In 02141 Computer Science Modelling you will learn how to formally describe syntax and semantics of programming languages.
- During the course, we will make such descriptions for reference, but not in all details formally. You will not be asked in the exam to describe syntax or semantics of F#.

The Syntax unlocked so far (1/3):

Constants

- `int` constants like $0, 1, -1, \ldots$
- `float` constants like 0.0, 3.14, ...
- `string` constants like "*Hello*"
- `bool` constants *true* and *false*

Identifiers

are programmer-chosen names for variables and functions, like `x7` or `exp`. They are sequences of alphanumberic characters (letters and digits), where the first must be a letter.

Keywords of F# like `let` also fulfill the definition of an identifier, but cannot be used as variable or function names, of course.

In compiler construction one typically formalizes concepts like constants and identifiers by regular expressions like `IDENT : [a-zA-Z][a-zA-Z0-9]*`
but we do not go into that in this course.

The Syntax unlocked so far (2/3):

Expressions

An expression can be any of the following:

- A constant
- An identifier
- If e, e_1, ... e_n are expressions and $\oplus$ is an infix operator (like $+$, $*$, $<$, $=$,...), then also the following are expressions:
  - ★ $e_1 \oplus e_2$
  - ★ `if` e `then` e_1 `else` e_2
  - ★ (e)
  - ★ $e_1\ e_2$
  - ★ $(e_1, \ldots, e_n)$

Examples:

- $3 * \mathtt{x} + 7$ This is equivalent to $(3 * \mathtt{x}) + 7$ (operator $*$ binds stronger than $+$)
- `sqrt 2` This is normal function application
  `sqrt(2)` is also correct, but not necessary.
- `sqrt 2` $*$ `sqrt 2` This is equivalent to (`sqrt 2`) $*$ (`sqrt 2`) (standard function application binds stronger than infix operators)
- `x` $*$ `if x` > 0 `then 3 else` -1

The Syntax unlocked so far (2/3):

Expressions

An expression can be any of the following:

- A constant
- An identifier
- If e, e_1, ... e_n are expressions and $\oplus$ is an infix operator (like $+$, $*$, $<$, $=$,...), then also the following are expressions:
  - ★ $e_1 \oplus e_2$
  - ★ `if` e `then` e_1 `else` e_2
  - ★ (e)
  - ★ $e_1\ e_2$
  - ★ $(e_1, \ldots, e_n)$

Note that this is an inductive definition:

- it defines some atomic expressions (constants, identifiers)
- and how larger expressions can be composed from smaller ones.

In compiler construction, one typically uses context-free grammars to express such inductive definitions, e.g.,

expr ::= *CONST* | *IDENT* | *expr INFIXOP expr* | `if` *expr* `then` *expr* `else` *expr* | ...

but we do not go into that in this course.

The Syntax unlocked so far (3/3):

Declarations

If e is an expression and f and $x_1, \ldots, x_n$ are identifiers, then the following are declarations:

- `let f = e`
- `let f (x1, ..., xn) = e`
- `let rec f = e`
- `let rec f (x1, ..., xn) = e`

Each of these define a function or value f by the expression e where

- Keyword `rec` must be used, if the definition is recursive, i.e., if f occurs in e.
- In the cases with $(\mathtt{x}_1, \ldots, \mathtt{x}_n)$, f is a function that takes an n-tuple as input and e may contain the x_i.

Example

```
let rec pow (x,n) = if n=0 then 1.0 else x * pow (x,n-1)
```

The Syntax unlocked so far (3/3):

Declarations

If e is an expression and f and $x_1, \ldots, x_n$ are identifiers, then the following are declarations:

- `let f = e`
- `let f (x1, ..., xn) = e`
- `let rec f = e`
- `let rec f (x1, ..., xn) = e`

Each of these define a function or value f by the expression e where

- Keyword `rec` must be used, if the definition is recursive, i.e., if f occurs in e.
- In the cases with $(\mathtt{x_1}, \ldots, \mathtt{x_n})$, f is a function that takes an n-tuple as input and e may contain the x_i.

Example

```
let rec pow (x,n) = if n=0 then 1.0 else x * pow (x,n-1)
```

That is the syntax we unlocked with the first example!
This is already Turing complete (with unbounded memory).

Patterns

We had also seen pattern matching as a way to describe case distinction nicely, for instance:

```
let rec power(x,n) =
   match (x,n) with
   | (_,0) -> 1.0                    (* 1 *)
   | (x,n) -> x * power(x,n-1)    (* 2 *)
```

Patterns

A pattern can be any of the following:

- A constant
- An identifier
- The wildcard _
- If $p_1, \ldots p_n$ are patterns, then also $(p_1, \ldots, p_n)$ is a pattern.

Expressions

An expression can also be:

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

if $p_1, \ldots, p_n$ are patterns and $e, e_1, \ldots, e_n$ are expressions.

Unlocking: we will extend the concepts of expressions, patterns, and declarations, during the course with every feature we discuss.

Outline

1. Recap: What we have seen of F# so far
2. **Last Week's Exercises**
3. Intro to Types
4. Intro to Higher-Order Functions
5. Lists
6. On functional decomposition
7. Exercises

Exercises

Recursive Functions:

- Hansen and Rischel Book (HR):
  Exercise 1.4: Define a function f with $f(n) = 1 + 2 + \ldots + n$.
  Exercise 1.6: Define a function *sum* such that
  $sum(m, n) = m + (m + 1) + ... + (m + n - 1) + (m + n)$
  Exercise 2.8: Define $\binom{n}{k}$ where $0 \leq k \leq n$, according to the definition:

$$\binom{n}{0} = \binom{n}{n} = 1 \qquad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Outline

1. Recap: What we have seen of F# so far
2. Last Week's Exercises
3. **Intro to Types**
4. Intro to Higher-Order Functions
5. Lists
6. On functional decomposition
7. Exercises

Introduction to Types

F# is a typed language:

- Every value is supposed to have a type (like `int`).
- There are typing rules that specify what is a type-correct expression, declaration, or pattern.
  - ★ This is a restriction on the syntactically correct expressions
  - ★ e.g. `3*3.1` is syntactically correct, but a type error (multiplying an `int` with a `float`).
- You (normally) do not specify types, but the F# interpreter or compiler will try to infer them for you.
- When F# tools report a type error, it almost always represents a real problem in your program: you specified something you don't want.

Types

Types

A type can be any of the following:

- Basic types: `int`, `float`, `bool`, `string` ...
- Tuples: $\tau_1 \star \ldots \star \tau_n$ where the τ_i are already types
- Functions: $\tau_1 \rightarrow \tau_2$ where τ_1 and τ_2 are types
- Later we also have type variables and type constructors

For instance we have `power: float*int -> float`

Generally $e : \tau$ means expression e is of type τ.

Basic Typing Rules

N.B.: Defining a typing system is involved, we simplify matters here.

Constants

Constants have basic types that are recognized directly:

- e.g. `3:int`, `3.14:float`, `true:bool` ...

Built-in Infix Functions

if	then
e_1 : `int` e_2 : `int`	$e_1 + e_2$: `int`
e_1 : `float` e_2 : `float`	$e_1 + e_2$: `float`
...	
e_1 : `int` e_2 : `int`	$e_1 = e_2$: `bool`
e_1 : `int` e_2 : `int`	$e_1 < e_2$: `bool`
...	

Variables/Identifiers

Usually type not given. It is the task of type inference to figure out a type that fits. E.g. in $x + 3.1$ we have that $3.1 : \mathit{float}$ and $+$ works only if both arguments have the same type, so $x : \mathit{float}$.

Basic Typing Rules

Conditionals

The expression `if` e_0 `then` e_1 `else` e_2 is only type correct if:

- $e_0 : \texttt{bool}$
- There is some type τ such that both $e_1 : \tau$ and $e_2 : \tau$

The expression has then type τ.

Example: `if n>0 then 1.0 else x`
Is this type correct, and what types must `n` and `x` have?

Tuples

For expressions $e_1 : \tau_1, \ldots, e_n : \tau_n$:
$(e_1, \ldots, e_n) : \tau_1 \star \ldots \star \tau_n$

Basic Typing Rules

Patterns

Typing of patterns works just like typing of expressions where a wildcard _ is replaced with a fresh variable name.

Pattern Matching

The expression

$$\begin{array}{l}\texttt{match } e \texttt{ with} \\ \quad \mid p_1 \rightarrow e_1 \\ \quad \ldots \\ \quad \mid p_n \rightarrow e_n\end{array}$$

is type correct if the following holds:

- The expression e must have the same type as the patterns:
  There is some type τ_0 such that $e : \tau_0$ and $p_1 : \tau_0, \ldots p_n : \tau$
- All the results for each case must have the same type:
  There is some type τ such that $e_1 : \tau, \ldots, e_n : \tau$

The entire match expression has then type τ.

Basic Typing Rules

Declarations

If $e : \tau$ for some type τ, then the declaration

$$\texttt{let rec } v = e$$

implies that also $v : \tau$.
If $e : \tau$ for some type τ and $p : \tau_0$ then

$$\texttt{let rec } f\ p = e$$

implies that $f : \tau_0 \to \tau$

In recursive function declarations like `power`, the type inference has to figure out types τ and τ_0 so that this rule is satisfied.

Example: Infer the type of power

```
let rec power (x,n) =
  match n with
  | 0 -> 1.0
  | _ -> x * power(x,n-1)
```

Outline

1. Recap: What we have seen of F# so far
2. Last Week's Exercises
3. Intro to Types
4. **Intro to Higher-Order Functions**
5. Lists
6. On functional decomposition
7. Exercises

Higher-Order Functions

Idea

Functions are values

- In conventional languages, often only data can be argument or result of a function (e.g., `int`, or `int-array`).
- In functional languages, also functions (computations) can be argument or result of a function.
- We call a function higher-order, if it has a function as argument or result.

Function Expressions

Let x be a variable of type τ_0 and e be an expression of type τ. Then also

$$\texttt{fun}\ x\ \texttt{->}\ e$$

is an expression that defines a function that takes an x and returns e. Thus, the resulting expression is of type $\tau_0 \to \tau$.

Example: what is the type of this?

```
let rec powerC =
  fun x ->
    fun n ->
      match n with
      | 0 -> 1.0
      | _ -> x * ( powerC x (n-1) )
```

For comparison:

```
let rec power (x,n) =
  match n with
  | 0 -> 1.0
  | _ -> x * power(x,n-1)
```

Function Expressions

Let x be a variable of type τ_0 and e be an expression of type τ. Then also

$$\texttt{fun}\ x\ \texttt{->}\ e$$

is an expression that defines a function that takes an x and returns e. Thus, the resulting expression is of type $\tau_0 \to \tau$.

Example: what is the type of this?

```
let rec powerC =
  fun x ->
    fun n ->
      match n with
      | 0 -> 1.0
      | _ -> x * ( powerC x (n-1) )

power  : float *  int -> float
powerC : float -> int -> float
```

For comparison:

```
let rec power (x,n) =
  match n with
  | 0 -> 1.0
  | _ -> x * power(x,n-1)
```

Some Syntactic Sugar

Since we often need this we have some syntactic sugar, i.e., abbreviating notation:

Function Declaration

$$\texttt{let rec } f\ p_1\ \ldots\ p_n = e$$

is an abbreviation for:

$$\texttt{let rec } f = \texttt{fun } p_1 \texttt{ -> } \ldots \texttt{ fun } p_n \texttt{ -> } e$$

Thus we can simply write

```
let rec powerC x n =
   match n with
   | 0 -> 1.0
   | _ -> x * ( powerC x (n-1) )

power  : float *  int -> float
powerC : float -> int -> float
```

For comparison:

```
let rec power (x,n) =
  match n with
  | 0 -> 1.0
  | _ -> x * power(x,n-1)
```

But what's the advantage of this?

Comparison: tuple vs. higher-order function

```
// powerC: float ->  (int -> float)
let rec powerC x n =
  match n with
  | 0 -> 1.0
  | _ -> x * ( powerC x (n-1) )
```

Comparison: tuple vs. higher-order function

```
// powerC: float ->  (int -> float)
let rec powerC x n =
  match n with
  | 0 -> 1.0
  | _ -> x * ( powerC x (n-1) )
```

- the argument of powerC is the base x

Comparison: tuple vs. higher-order function

```
// powerC: float ->  (int -> float)
let rec powerC x n =
  match n with
  | 0 -> 1.0
  | _ -> x * ( powerC x (n-1) )
```

- the argument of powerC is the base x
- and `powerC` x is the function that maps exponent n to x^n

Comparison: tuple vs. higher-order function

```
// powerC: float ->  (int -> float)
let rec powerC x n =
  match n with
  | 0 -> 1.0
  | _ -> x * ( powerC x (n-1) )
```

- the argument of powerC is the base x
- and `powerC` x is the function that maps exponent n to x^n

The function may be evaluated in *stages*:

```
let pow2 = powerC 2.0;;
pow2 3;;
val it : float = 8.0
pow2 4;;
val it : float = 16.0
```

Higher-Order Functions

General Idea

- Instead of a function that takes an n-tuple of arguments

$$f : \tau_1 \star \ldots \tau_n \rightarrow \tau$$

- you can have a higher-order function that evaluates in n stages of one argument:

$$fC : \tau_1 \rightarrow \tau_2 \rightarrow \ldots \tau_n \rightarrow \tau$$

- Idea due to Moses Schönfinkel and Haskell Curry
  - ★ Hence they are called schönfinkeled or curried functions.

This allows often for very elegant programs, so typically people define functions curried in functional languages.

Example:

```
let cube x = powerC x 3
```

Outline

1. Recap: What we have seen of F# so far
2. Last Week's Exercises
3. Intro to Types
4. Intro to Higher-Order Functions
5. **Lists**
6. On functional decomposition
7. Exercises

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[\,]$ is called the empty list)

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[\,]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]

[(1,true); (3,true)];;
val it : (int * bool) list = [(1, true); (3, true)]
```

Lists

A list is a finite sequence of elements having the same type:

$[v_1; \ldots; v_n]$ ($[]$ is called the empty list)

```
[2;3;6];;
val it : int list = [2; 3; 6]

["a"; "ab"; "abc"; ""];;
val it : string list = ["a"; "ab"; "abc"; ""]

[sin; cos];;
val it : (float->float) list = [<fun:...>; <fun:...>]

[(1,true); (3,true)];;
val it : (int * bool) list = [(1, true); (3, true)]

[[]; [1]; [1;2]];;
val it : int list list = [[]; [1]; [1; 2]]
```

List constructors

The formal definition of lists:

Lists

A list can be **constructed** as follows:

- [] (the empty list)
- If *e* is an expression and *l* is a list , then also *e* :: *l* is a list .

Lists are also expressions.

List constructors

The formal definition of lists:

Lists

A list (with elements of type τ) can be **constructed** as follows:

- [] (the empty list)
- If e is an expression (of type τ) and l is a list (with elements of type τ), then also
  e :: l is a list with elements of type τ .

Lists are also expressions.

Lists are homogenous, i.e., all elements are of the same (arbitrary) type τ.
The list type is thus called τ-list.

List constructors

The formal definition of lists:

Lists

A list (with elements of type τ) can be **constructed** as follows:

- [] (the empty list)
- If e is an expression (of type τ) and l is a list (with elements of type τ), then also
  $e :: l$ is a list with elements of type τ .

Lists are also expressions.

Lists are homogenous, i.e., all elements are of the same (arbitrary) type τ.
The list type is thus called τ-list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a syntactic sugar for $x_1 :: x_2 :: \ldots :: x_n :: []$.

List constructors

The formal definition of lists:

Lists

A list (with elements of type τ) can be **constructed** as follows:

- [] (the empty list)
- If e is an expression (of type τ) and l is a list (with elements of type τ), then also
  $e :: l$ is a list with elements of type τ .

Lists are also expressions.

Lists are homogenous, i.e., all elements are of the same (arbitrary) type τ.
The list type is thus called τ-list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a syntactic sugar for $x_1 :: x_2 :: \ldots :: x_n :: []$.

The constructor :: is pronounced *cons*

List constructors

The formal definition of lists:

Lists

A list (with elements of type τ) can be **constructed** as follows:

- [] (the empty list)
- If e is an expression (of type τ) and l is a list (with elements of type τ), then also
  $e :: l$ is a list with elements of type τ .

Lists are also expressions.

Lists are homogenous, i.e., all elements are of the same (arbitrary) type τ.
The list type is thus called τ-list.

The notation $[x_1; x_2; \ldots; x_n]$ is just a syntactic sugar for $x_1 :: x_2 :: \ldots :: x_n :: []$.

The constructor :: is pronounced *cons*

For a non-empty list $[x_1; x_2; \ldots; x_n]$, $n \geq 1$, we call

- x_1 the *head* and
- $[x_2; \ldots; x_n]$ the *tail*.

There are library functions `List.head` and `List.tail`.

Lists for Pattern Matching

Pattern matching (for Lists)

Let p_1 and p_2 be patterns, then also the following are patterns:

- []
  matching only the empty list
- $p_1 :: p_2$
  matching only a list with at least one element, where
  - ★ the head matches p_1 (of some type τ)
  - ★ and the tail matches p_2 (of type τ *list*)

Recursion on lists – a simple example

$$\texttt{suml}\ [x_1\,;x_2\,;\ldots;x_n] = \sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n = x_1 + \sum_{i=2}^{n} x_i$$

Recursion on lists – a simple example

$$\texttt{suml}\ [x_1\,;x_2\,;\ldots;x_n] = \sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n = x_1 + \sum_{i=2}^{n} x_i$$

Constructors are used in list patterns

```
let rec suml xs =
  match xs with
  | [] -> 0
  | x::tail -> x + suml tail

val suml : int list -> int
```

```
    suml [1;2]
 ⇝  1 + suml [2]          (x ↦ 1 and tail ↦ [2])
 ⇝  1 + (2 + suml [])     (x ↦ 2 and tail ↦ [])
 ⇝  1 + (2 + 0)           (the pattern [] matches the value [])
 ⇝  1 + 2
 ⇝  3
```

Recursion follows the structure of lists

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
  match xs with
  | [] -> []
  | x::tail when x=y -> remove y tail
  | x::tail -> x::remove y tail
```

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
  match xs with
  | [] -> []
  | x::tail when x=y -> remove y tail
  | x::tail -> x::remove y tail
```

List elements can be of any type that supports equality

```
remove :  'a -> 'a list    -> 'a list when 'a : equality
```

A polymorphic list function (I)

The function remove y xs gives the list obtained from xs by deleting every occurrence of y, e.g. `remove` $2\ [1;2;0;2;7] = [1;0;7]$.

Recursion is following the structure of the list:

```
let rec remove y xs =
  match xs with
  | [] -> []
  | x::tail when x=y -> remove y tail
  | x::tail -> x::remove y tail
```

List elements can be of any type that supports equality

```
remove :  'a -> 'a list    -> 'a list when 'a : equality
```

- 'a is a *type variable*. I usually prefer to write $\alpha, \tau, \ldots$
- 'a : equality is a type constraint
  - ★ we can only apply `remove` to α lists, if equality is defined on α, e.g. *int*, *string*, ... but not functions.

The F# system infers the most general type for remove

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of 'a:

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of `'a`:

Instantiating `'a` with `int`:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of `'a`:

Instantiating `'a` with `int`:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

Instantiating `'a` with `int list`:

```
remove [2] [[2;1]; [2]; [0;1]; [2]; [5;6;7]];;
val it : int list list = [[2; 1]; [0; 1]; [5; 6; 7]]
```

A polymorphic list function (II)

- A type containing type variables is called a polymorphic type
- The remove function is called a polymorphic function.

```
remove : 'a -> 'a list -> 'a list when 'a : equality
```

The function has many forms, one for each instantiation of 'a:

Instantiating 'a with int:

```
remove 2 [1; 2; 0; 2; 7];;
val it : int list = [1; 0; 7]
```

Instantiating 'a with int list:

```
remove [2] [[2;1]; [2]; [0;1]; [2]; [5;6;7]];;
val it : int list list = [[2; 1]; [0; 1]; [5; 6; 7]]
```

Notice that -> associates to the right:

```
'a -> 'a list -> 'a list  means 'a -> ('a list -> 'a list)
```

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

Exploiting structured patterns: the isPrefix function

The function `isPrefix` *xs ys* tests whether the list *xs* is a prefix of the list *ys*, for example:

$$\texttt{isPrefix}\ [1;2;3]\ [1;2;3;8;9] \quad = \quad \text{true}$$
$$\texttt{isPrefix}\ [1;2;3]\ [1;2;8;3;9] \quad = \quad \text{false}$$

The function is declared as follows:

```
let rec isPrefix xs ys =    // Each clause expresses
  match (xs,ys) with        // a natural property:
  | ([],_) -> true          // [] is prefix of any list
  | (_,[]) -> false         // non-empty list is not prefix of []
  | (x::xtail,y::ytail) ->  // non-empty is prefix of non-empty
      x=y &&                // if first element equal and
      isPrefix xtail ytail  // rests are in prefix relation

isPrefix [1;2;3] [1;2];;
val it : bool = false
```

Summary

- Lists

Summary

- Lists
- Polymorphism

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values
- Constructors used in expressions to compose structured values

Summary

- Lists
- Polymorphism
- Constructors (:: and [] for lists)
- Patterns
- Recursion on the structure of lists
- Constructors used in patterns to decompose structured values
- Constructors used in expressions to compose structured values

Outline

1. Recap: What we have seen of F# so far
2. Last Week's Exercises
3. Intro to Types
4. Intro to Higher-Order Functions
5. Lists
6. **On functional decomposition**
7. Exercises

Functional decomposition

A simple technique when solving a complex problem is

- to partition it into smaller well-defined parts and, thereafter,
- to compose these parts to solve the original problem.

Functional decomposition

A simple technique when solving a complex problem is

- to partition it into smaller well-defined parts and, thereafter,
- to compose these parts to solve the original problem.

The main goal is that a program is constructed by

- combining simple well-understood pieces

Functional decomposition

A simple technique when solving a complex problem is

- to partition it into smaller well-defined parts and, thereafter,
- to compose these parts to solve the original problem.

The main goal is that a program is constructed by

- combining simple well-understood pieces

Invent useful helper functions

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Insertion in an ordered list is easy:

```
(* insert:'a -> 'a list -> 'a list when 'a : comparison *)
```

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Insertion in an ordered list is easy:

```
(* insert:'a -> 'a list -> 'a list when 'a : comparison *)

let rec insert x ys =
  match ys with
  | [] -> [x]
  | y::_ when x <= y -> x::ys
  | y::ytail -> y::insert x ytail
```

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Insertion in an ordered list is easy:

```
(* insert:'a -> 'a list -> 'a list when 'a : comparison *)

let rec insert x ys =
  match ys with
  | [] -> [x]
  | y::_ when x <= y -> x::ys
  | y::ytail -> y::insert x ytail
```

Insertion sort can now easily be implemented:

```
let rec sort xs =
  match xs with
  | [] -> []
  | x::xtail -> insert x (sort xtail)
```

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Insertion in an ordered list is easy:

```
(* insert:'a -> 'a list -> 'a list when 'a : comparison *)

let rec insert x ys =
  match ys with
  | [] -> [x]
  | y::_ when x <= y -> x::ys
  | y::ytail -> y::insert x ytail
```

Insertion sort can now easily be implemented:

```
let rec sort xs =
  match xs with
  | [] -> []
  | x::xtail -> insert x (sort xtail)
```

- small comprehensible programs

Functional decomposition: Example 1

Sorting a list: `sort: 'a list -> 'a list when 'a : comparison`.

Insertion in an ordered list is easy:

```
(* insert:'a -> 'a list -> 'a list when 'a : comparison *)

let rec insert x ys =
  match ys with
  | [] -> [x]
  | y::_ when x <= y -> x::ys
  | y::ytail -> y::insert x ytail
```

Insertion sort can now easily be implemented:

```
let rec sort xs =
  match xs with
  | [] -> []
  | x::xtail -> insert x (sort xtail)
```

- small comprehensible programs

Notice: there are better sorting algorithms than insertion sort

Idea: mergesort

Idea mergesort:

- Use recursion for sorting.
- To sort a list, one could split it into two smaller parts of half the size and sort the parts recursively.
- Then we just need to merge the two sorted parts into one sorted list.
- What is the base case of the recursion, i.e., where no further recursion is necessary?
- What's the runtime of mergesort?

Functional decomposition: Example 2

Consider multiplication of polynomials:

$$\begin{aligned} 0 \cdot Q(x) &= 0 \\ (a_0 + a_1 \cdot x + \ldots + a_n \cdot x^n) \cdot Q(x) &= a_0 \cdot Q(x) + x \cdot \left((a_1 + a_2 \cdot x + \ldots + a_n \cdot x^{n-1}) \cdot Q(x)\right) \end{aligned}$$

Functional decomposition: Example 2

Consider multiplication of polynomials:

$$\begin{aligned} 0 \cdot Q(x) &= 0 \\ (a_0 + a_1 \cdot x + \ldots + a_n \cdot x^n) \cdot Q(x) &= a_0 \cdot Q(x) + x \cdot \left((a_1 + a_2 \cdot x + \ldots + a_n \cdot x^{n-1}) \cdot Q(x)\right) \end{aligned}$$

Idea

Represent polynomes by a list of coefficients:

`type Poly = int list` defines a new type `Poly`

- $[1; 2; 3]$ represents $1 + 2x + 3x^2$
- $[2; 0; 0; 1]$ represents $2 + x^3$
- General: $[a_0; \ldots; a_n]$ for $a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$

Functional decomposition: Example 2

Consider multiplication of polynomials:

$$\begin{aligned} 0 \cdot Q(x) &= 0 \\ (a_0 + a_1 \cdot x + \ldots + a_n \cdot x^n) \cdot Q(x) &= a_0 \cdot Q(x) + x \cdot \left((a_1 + a_2 \cdot x + \ldots + a_n \cdot x^{n-1}) \cdot Q(x)\right) \end{aligned}$$

Idea

Represent polynomes by a list of coefficients:
`type Poly = int list` defines a new type `Poly`

- $[1; 2; 3]$ represents $1 + 2x + 3x^2$
- $[2; 0; 0; 1]$ represents $2 + x^3$
- General: $[a_0; \ldots; a_n]$ for $a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$

Invent suitable auxiliary functions

- `mulC: int -> Poly -> Poly` — for multiplying a polynome by a constant
  e.g. `mulC 3 [1;2;3] = [3;6;9]`
- `mulX: Poly -> Poly` — for multiplying a polynome $P(x)$ by its variable x
  e.g. `mulX [1;2;3] = [0;1;2;3]`
- `add: Poly -> Poly -> Poly` — for adding two polynomes
  e.g. `add [1;2;3] [0;1] = [1;3;3]`

With these helpers define `mul: Poly -> Poly -> Poly` by the equation above.
e.g. `mul [2;3;0;1] [1;2;3]= [2;7;12;10;2;3]`

Invent helper function(s): Blackboard exercise

Declare a function `sumProd: int list -> int*int`:

```
sumProd [x0;x1; ...;xn-1]
              =   ( x0 + x1 + ... + xn-1 , x0 * x1 * ... * xn-1 )
sumProd []    =   (0,1)
```

Outline

1. Recap: What we have seen of F# so far
2. Last Week's Exercises
3. Intro to Types
4. Intro to Higher-Order Functions
5. Lists
6. On functional decomposition
7. **Exercises**

Exercises

- HR 1.5: the Fibonacci numbers

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \text{ for } n > 1 \end{aligned}$$

- HR 2.2: $\texttt{pow(s,n)} = \underbrace{s \cdot s \cdot \ldots \cdot s}_{n}$ for string s and int n.
- HR 4.7: `multiplicity x xs` – compute the number of times `x` occurs in `xs`.
- Polynome exercise from slide 36 ($\approx$ HR 4.22).
- Merge sort: idea explained in class on blackbord
  Define helper functions (they are all in the List library, but easy to write yourself):
  - ★ `length: 'a list->int` length of a list
    e.g. `length [1;2;3] = 3`
  - ★ `split: int->'a list->('a list,'a list)` split a list into two after a given number of elements
    e.g. `split 2 [1;2;3;4;5] = ([1;2],[3;4;5])`
  - ★ `merge: ('a list,'a list) -> 'a list` merge a pair of sorted lists into one sorted list.

  Put it together into mergesort.